

Documento maestro — API evolutiva de análisis acústico para investigación en health care

0. Propósito del documento

Este documento define, con nivel de especificación funcional y técnica, la evolución incremental de una API existente de análisis de audio.

El objetivo no es reconstruir el proyecto, sino complementar la API actual con un módulo desacoplado que produzca métricas acústicas útiles para:

- investigación,
- dashboards,
- análisis longitudinal,
- exportación científica,
- preparación de datos para machine learning,
- futuros casos de uso en health care no diagnóstico.

El documento está diseñado para ser leído por Codex como especificación de implementación por fases. Cada fase incluye instrucciones específicas. Codex no debe interpretar este documento como un único prompt global, sino como una hoja de ruta incremental.

1. Principio rector del producto

La plataforma debe ser entendida como una API de generación de representaciones acústicas estructuradas.

No debe venderse ni implementarse como sistema diagnóstico.

Frase rectora

La plataforma es una infraestructura de análisis acústico para investigación, bienestar, dashboards exploratorios y generación de datos preparados para machine learning. No proporciona diagnóstico, tratamiento ni recomendaciones clínicas.

Qué se construye

Se construye una API que recibe audio, lo normaliza, lo divide en tramas, calcula métricas acústicas robustas y devuelve resultados estructurados en JSON.

Qué no se construye en esta fase

No se construye todavía:

- dashboard,
- frontend,

- app móvil,
 - modelo clínico,
 - clasificador diagnóstico,
 - sistema de decisión clínica,
 - entrenamiento deep learning,
 - inferencia con modelos pesados,
 - certificación médica.
-

2. Contexto técnico actual

El sistema ya existe.

Características conocidas:

- API existente,
- ejecución en Linux,
- despliegue mediante Docker,
- aproximadamente 8GB de RAM,
- endpoints ya funcionales,
- respuesta JSON ya definida,
- código inicial que debe conservarse.

Restricciones obligatorias

Codex debe respetar:

- No reconstruir el proyecto desde cero.
 - No eliminar endpoints existentes.
 - No romper la respuesta JSON actual.
 - No modificar la arquitectura base salvo que sea imprescindible.
 - No introducir dependencias pesadas sin justificación.
 - No usar GPU.
 - No introducir deep learning en las primeras fases.
 - No devolver matrices grandes dentro del JSON.
 - No generar artefactos pesados por defecto.
 - No guardar scalogramas o espectrogramas como imágenes en el flujo normal.
 - No añadir claims médicos o diagnósticos.
-

3. Motivación técnica según estado del arte

La literatura reciente sobre biomarcadores acústicos y audio biomédico muestra que las métricas útiles suelen agruparse en varios dominios:

1. calidad de señal,
2. energía y amplitud,
3. características temporales,
4. características espectrales,
5. características cepstrales,

6. características tiempo-frecuencia,
7. métricas longitudinales,
8. representaciones preparadas para machine learning.

En voz y biomarcadores digitales se emplean habitualmente medidas como F0, jitter, shimmer, MFCCs, energía, loudness, formantes, medidas espectrales y prosódicas. En sonidos respiratorios se utilizan métricas de tiempo, frecuencia, energía, entropía y wavelets. En uroflujometría acústica, la señal sonora se ha usado para estimar patrones de flujo a partir de la interacción del chorro con el agua, lo que justifica métricas temporales, energéticas y espectro-temporales.

Por tanto, la API debe devolver métricas interpretables, auditables y compactas. No debe limitarse a devolver imágenes ni únicamente valores globales.

4. Principio arquitectónico

La evolución debe realizarse mediante un módulo nuevo, desacoplado y opcional llamado:

```
analysis_engine/
```

Este módulo debe enriquecer la respuesta actual de la API añadiendo una nueva clave:

```
"analysis_engine": {}
```

La respuesta existente debe mantenerse intacta.

Patrón de integración esperado

```
existing_result = existing_audio_analysis(...)

from analysis_engine.orchestrator import run_analysis_engine

analysis_engine_result = run_analysis_engine(
    audio_input=audio_input,
    sample_rate=sample_rate,
    original_format=original_format,
    config=config,
)

existing_result["analysis_engine"] = analysis_engine_result

return existing_result
```

5. Arquitectura del módulo

Crear, sin alterar la arquitectura principal del proyecto, el siguiente módulo:

```
analysis_engine/
├── __init__.py
├── config.py
├── audio_io.py
├── validation.py
├── framing.py
├── serialization.py
├── stage0_quality.py
├── stage1_basic.py
├── stage2_temporal.py
├── stage3_spectral.py
├── stage4_cepstral.py
├── stage5_timefreq.py
├── stage6_ml_ready.py
├── stage7_research_export.py
├── orchestrator.py
└── schemas.py
```

La numeración de etapas refleja la prioridad de construcción.

6. Modos de funcionamiento

La API debe distinguir dos modos.

6.1. Modo normal

Modo por defecto.

Características:

- devuelve JSON ligero,
- pensado para API, dashboard y almacenamiento,
- no guarda artefactos pesados,
- no exporta `.mat` automáticamente,
- no devuelve audio bruto,
- no devuelve matrices CWT/STFT completas,
- debe ser rápido,
- debe funcionar con 8GB RAM.

6.2. Modo research

Modo explícito y bajo demanda.

Características:

- genera archivos científicos,
- permite exportar `.mat`, `.csv` y `.json`,
- puede incluir audio normalizado,
- puede incluir tramas,
- puede incluir features globales y por trama,
- puede incluir información compacta tiempo-frecuencia,
- debe tener límites de tamaño,
- no debe ejecutarse en cada análisis normal.

7. Formatos de audio aceptados

La API debe aceptar:

```
.wav  
.webm  
.mp3  
.m4a  
.ogg  
.flac  
.aac
```

Prioridad

Formato	Prioridad	Comentario
<code>.wav</code>	Alta	Preferente para investigación.
<code>.webm</code>	Alta	Importante para grabación desde navegador.
<code>.flac</code>	Media	Sin pérdida, útil en investigación.
<code>.ogg</code>	Media	Común en navegador/Linux.
<code>.m4a</code>	Media	Frecuente en móviles.
<code>.mp3</code>	Media	Común, pero con pérdida.
<code>.aac</code>	Baja	Aceptar si ffmpeg está disponible.

Dependencias

Preferencias:

- `soundfile` para `.wav` y `.flac`,
- `librosa` si ya está disponible,
- `ffmpeg` solo si el contenedor lo soporta o si ya existe en el proyecto.

No introducir dependencias pesadas si el proyecto actual ya resuelve la lectura de audio.

8. Normalización interna

Todo audio debe convertirse internamente a:

```
sample_rate: 16000 Hz por defecto
canales: mono
dtype: float32
amplitud: [-1.0, 1.0]
```

Configuración recomendada

```
AUDIO_CONFIG = {
    "target_sample_rate": 16000,
    "mono": True,
    "dtype": "float32",
    "normalize_amplitude": True,
    "max_audio_duration_seconds": 180,
    "accepted_extensions": [".wav", ".webm", ".mp3", ".m4a", ".ogg",
    ".flac", ".aac"],
}
```

Metadatos mínimos de entrada

El JSON debe registrar:

```
"input_audio": {
    "original_format": "webm",
    "internal_sample_rate": 16000,
    "channels": 1,
    "duration_seconds": 34.7,
    "normalization_applied": true
}
```

9. Procesamiento por tramas

El procesamiento por tramas es obligatorio.

La API debe calcular métricas:

- globales,
- por trama,
- agregadas longitudinalmente.

Configuración sugerida

```
FRAMING_CONFIG = {  
    "enabled": True,  
    "frame_duration_seconds": 2.0,  
    "hop_duration_seconds": 1.0,  
    "allow_overlap": True,  
    "auto_frame_duration": True,  
    "min_frame_duration_seconds": 0.5,  
    "max_frame_duration_seconds": 10.0,  
    "max_frames": 300,  
}
```

Regla de autoajuste

Duración del audio	Frame	Hop
0–15 s	1 s	0.5 s
15–60 s	2 s	1 s
60–180 s	3 s	1.5 s
>180 s	rechazar o truncar	—

Justificación

Tramas de 2 segundos son un buen punto inicial porque:

- reducen memoria,
- permiten curvas temporales suaves,
- generan suficientes muestras para dashboards,
- preparan secuencias para ML,
- evitan procesar el audio completo con operaciones caras.

10. Estructura general del JSON

La clave `analysis_engine` debe seguir esta estructura:

```
{  
  "analysis_engine": {  
    "version": "0.1.0",  
    "status": "success",  
    "mode": "normal",  
    "input_audio": {},  
    "framing": {},  
    "quality": {},  
    "global_features": {},  
    "frame_features": [],  
  },  
}
```

```
    "temporal_summary": {},
    "spectral_summary": {},
    "cepstral_summary": {},
    "time_frequency_summary": {},
    "ml_ready": {},
    "dashboard_ready": {},
    "exports": {},
    "errors": []
  }
}
```

11. Manejo de errores

Una etapa puede fallar sin romper la API completa.

Estados permitidos:

```
success
partial_success
failed
skipped
```

Ejemplo:

```
{
  "status": "partial_success",
  "errors": [
    {
      "stage": "stage5_timefreq",
      "message": "Time-frequency analysis skipped due to resource limits."
    }
  ]
}
```

FASE 0 — Validación, carga, normalización y calidad de señal

Objetivo

Construir la base robusta de la API acústica. Antes de extraer features, se debe asegurar que el audio es válido, usable y normalizado.

Archivos

```
analysis_engine/audio_io.py
analysis_engine/validation.py
analysis_engine/stage0_quality.py
analysis_engine/serialization.py
```

Métricas de calidad

Calcular:

- duration_seconds,
- sample_rate_original,
- sample_rate_internal,
- channels_original,
- is_mono_internal,
- clipping_ratio,
- silence_ratio,
- active_ratio,
- estimated_noise_floor,
- snr_estimate,
- peak_amplitude,
- mean_amplitude,
- dc_offset,
- valid_audio,
- quality_flag.

quality_flag

Valores sugeridos:

```
good
low_snr
mostly_silent
clipped
too_short
too_long
invalid
```

JSON esperado

```
"quality": {
  "valid_audio": true,
  "quality_flag": "good",
  "duration_seconds": 34.7,
  "clipping_ratio": 0.001,
  "silence_ratio": 0.12,
```

```
"active_ratio": 0.88,  
"snr_estimate": 18.2,  
"peak_amplitude": 0.82,  
"dc_offset": 0.001  
}
```

Instrucción específica para Codex — Fase 0

Implement Phase 0 of the `analysis_engine` module.

Focus only on API-side audio validation, loading, normalization, framing preparation, quality metrics, and safe JSON serialization.

Do not implement advanced features yet.

Do not modify existing endpoints.

Do not break the existing JSON response.

Implement:

- `config.py`
- `audio_io.py`
- `validation.py`
- `framing.py`
- `serialization.py`
- `stage0_quality.py`
- minimal orchestrator integration

Requirements:

- accept configured audio formats,
- normalize to mono, 16 kHz, float32,
- enforce max duration,
- compute quality metrics,
- split into frames,
- return JSON-serializable values only,
- no NumPy arrays in JSON,
- include partial error handling.

Add tests for:

- accepted formats,
- unsupported formats,
- normalization output dtype,
- frame splitting,
- quality flags,
- JSON serializability.

FASE 1 — Features básicas de energía y tiempo

Objetivo

Calcular métricas de bajo coste y alta utilidad. Estas métricas son la base del dashboard y del futuro vector ML.

Archivo

```
analysis_engine/stage1_basic.py
```

Métricas globales y por trama

Calcular:

- rms_mean,
- rms_std,
- rms_min,
- rms_max,
- rms_median,
- short_time_energy_mean,
- short_time_energy_std,
- zero_crossing_rate_mean,
- zero_crossing_rate_std,
- peak_count,
- crest_factor,
- dynamic_range_db,
- energy_entropy,
- active_duration_seconds.

Justificación

Estas métricas capturan intensidad, estabilidad, variabilidad y actividad. Son útiles para audio biomédico, voz, respiración y señales tipo flujo.

JSON esperado por trama

```
{
  "frame_index": 0,
  "start_time": 0.0,
  "end_time": 2.0,
  "basic_features": {
    "rms_mean": 0.031,
    "rms_std": 0.004,
    "rms_max": 0.12,
    "zero_crossing_rate_mean": 0.08,
```

```
"short_time_energy_mean": 0.002,  
"energy_entropy": 1.73  
}  
}
```

Instrucción específica para Codex — Fase 1

Implement Phase 1: basic time-domain and energy features.

Use normalized audio and frames from Phase 0.

Compute features globally and per frame.

Keep all outputs compact and JSON-ready.

Implement:

- stage1_basic.py
- orchestrator integration for Phase 1
- dashboard_ready fields for RMS and energy trends

Requirements:

- do not recompute loading/normalization,
- do not return arrays longer than configured limits,
- calculate stable scalar features,
- handle silent audio safely,
- keep execution lightweight.

Add tests for:

- normal audio,
- silent audio,
- short audio,
- frame-level feature schema,
- global feature schema.

FASE 2 — Dinámica temporal y patrones longitudinales dentro del audio

Objetivo

Transformar las métricas por trama en descriptores temporales útiles.

Archivo

```
analysis_engine/stage2_temporal.py
```

Métricas

Calcular:

- rms_trend_reduced,
- energy_trend_reduced,
- power_slope,
- power_intercept,
- power_r2,
- num_energy_peaks,
- peak_frame_indices,
- peak_times_seconds,
- time_to_peak_seconds,
- attack_time_seconds,
- decay_time_seconds,
- stability_index,
- variability_index,
- coefficient_of_variation,
- active_frame_ratio,
- silence_frame_ratio,
- temporal_asymmetry,
- early_energy_ratio,
- middle_energy_ratio,
- late_energy_ratio.

Justificación

Estas métricas permiten representar forma, evolución y estabilidad. Para sonidos asociados a procesos fisiológicos, la dinámica temporal suele ser más relevante que un único promedio global.

JSON esperado

```
"temporal_summary": {
  "power_slope": 0.012,
  "power_r2": 0.71,
  "num_energy_peaks": 5,
  "peak_times_seconds": [4.0, 8.0, 13.0],
  "time_to_peak_seconds": 4.0,
  "stability_index": 0.83,
  "variability_index": 0.21,
  "active_frame_ratio": 0.78,
  "silence_frame_ratio": 0.22,
  "early_energy_ratio": 0.20,
  "middle_energy_ratio": 0.55,
  "late_energy_ratio": 0.25
}
```

Instrucción específica para Codex — Fase 2

Implement Phase 2: temporal dynamics.

Use frame-level features from Phase 1.

Do not recompute audio features unnecessarily.

Focus on compact temporal descriptors useful for dashboards and future ML.

Implement:

- stage2_temporal.py
- trend reduction according to config
- peak detection on energy/RMS trends
- stability and variability metrics
- orchestrator integration

Requirements:

- limit returned trend points,
- handle very short signals,
- handle silent or nearly silent audio,
- produce dashboard_ready trend fields,
- keep all values JSON serializable.

Add tests for:

- increasing energy trend,
- decreasing energy trend,
- flat signal,
- silent signal,
- peak detection,
- trend length limits.

FASE 3 — Features espectrales compactas

Objetivo

Calcular métricas de frecuencia útiles sin devolver espectrogramas completos.

Archivo

```
analysis_engine/stage3_spectral.py
```

Métricas

Calcular globalmente y, si es viable, por trama:

- spectral_centroid_mean,

- spectral_centroid_std,
- spectral_bandwidth_mean,
- spectral_bandwidth_std,
- spectral_rolloff_85_mean,
- spectral_rolloff_95_mean,
- spectral_flatness_mean,
- spectral_contrast_mean,
- spectral_flux_mean,
- dominant_frequency,
- low_band_energy_ratio,
- mid_band_energy_ratio,
- high_band_energy_ratio,
- band_energy_entropy,
- power_spectral_density_summary.

Bandas iniciales sugeridas

Para audio general a 16 kHz:

```
low: 0-300 Hz
mid: 300-3000 Hz
high: 3000-8000 Hz
```

Para casos futuros se podrá parametrizar por dominio:

- voz,
- respiración,
- uroflujometría acústica,
- ambiente,
- otros.

JSON esperado

```
"spectral_summary": {
  "spectral_centroid_mean": 1350.4,
  "spectral_bandwidth_mean": 850.2,
  "spectral_rolloff_85_mean": 3100.0,
  "spectral_flatness_mean": 0.21,
  "dominant_frequency": 740.0,
  "energy_bands": {
    "low": 0.22,
    "mid": 0.57,
    "high": 0.21
  },
  "band_energy_entropy": 1.03
}
```

Instrucción específica para Codex — Fase 3

Implement Phase 3: compact spectral features.

Do not return full spectrograms.

Do not save images.

Compute scalar spectral metrics and compact band-energy summaries.

Implement:

- stage3_spectral.py
- configurable frequency bands
- global spectral summary
- optional frame-level spectral features if lightweight
- orchestrator integration

Requirements:

- avoid large arrays in JSON,
- use float32 where possible,
- handle short frames safely,
- handle low-energy audio safely,
- keep feature names stable.

Add tests for:

- sinusoidal signal with known dominant frequency,
- broadband noise,
- silent audio,
- band energy ratios sum approximately to 1,
- JSON serialization.

FASE 4 — Features cepstrales y de voz opcionales

Objetivo

Añadir features útiles para voz y audio biomédico general, especialmente MFCCs, sin convertir la API en una herramienta exclusivamente de voz.

Archivo

```
analysis_engine/stage4_cepstral.py
```

Métricas

Calcular:

- mfcc_mean_1_to_13,

- mfcc_std_1_to_13,
- delta_mfcc_mean_1_to_13 opcional,
- delta_mfcc_std_1_to_13 opcional,
- spectral_envelope_summary.

Métricas de voz opcionales

Solo si la configuración activa `voice_features`:

- estimated_f0_mean,
- estimated_f0_std,
- estimated_f0_min,
- estimated_f0_max,
- voiced_ratio,
- jitter_proxy,
- shimmer_proxy.

Importante: jitter y shimmer reales requieren estimación robusta de ciclos glotales. En esta API inicial se deben marcar como proxies si se calculan de forma aproximada.

JSON esperado

```
"cepstral_summary": {
  "mfcc_mean": [0.12, -0.03, 0.44],
  "mfcc_std": [0.02, 0.08, 0.11],
  "num_mfcc": 13,
  "voice_features_enabled": false
}
```

Instrucción específica para Codex — Fase 4

Implement Phase 4: cepstral features.

Add MFCC summaries useful for ML and research.

Do not make this API voice-only.

Voice-specific features must be optional and clearly marked.

Do not claim clinical meaning.

Implement:

- stage4_cepstral.py
- MFCC mean/std summaries
- optional voice feature block controlled by config
- orchestrator integration

Requirements:

- keep vectors short,
- use stable feature ordering,
- mark jitter/shimmer only as proxies if implemented,
- handle non-voice audio safely,

- keep output JSON serializable.

Add tests for:

- MFCC output length,
- missing/disabled voice features,
- short audio,
- silent audio,
- stable feature names.

FASE 5 — Features tiempo-frecuencia compactas

Objetivo

Extraer información tiempo-frecuencia avanzada sin generar imágenes ni devolver matrices grandes.

Archivo

```
analysis_engine/stage5_timefreq.py
```

Métricas

Calcular de forma opcional:

- wavelet_entropy,
- dominant_scale,
- scale_energy_distribution_reduced,
- time_frequency_concentration,
- frequency_centroid_timefreq,
- frequency_spread_timefreq,
- transient_index,
- modulation_energy_summary.

Restricciones

- Fase desactivada por defecto inicialmente.
- Máximo 64 escalas.
- Downsampling interno si es necesario.
- No devolver matriz CWT.
- No guardar imagen de scalograma.
- Procesar audio global o subconjunto representativo.

JSON esperado

```
"time_frequency_summary": {  
  "enabled": true,
```

```
"wavelet_entropy": 2.31,  
"dominant_scale": 45,  
"time_frequency_concentration": 0.67,  
"scale_energy_distribution_reduced": [0.03, 0.06, 0.12, 0.20]  
}
```

Instrucción específica para Codex — Fase 5

Implement Phase 5: compact time-frequency features.

This stage must be optional and disabled by default.

Extract compact features only.

Do not return CWT/STFT matrices.

Do not save scalogram images in normal mode.

Implement:

- stage5_timefreq.py
- config flags for enabling/disabling
- safe execution with `partial_success` behavior
- reduced feature summaries

Requirements:

- limit wavelet scales,
- optionally downsample before CWT,
- avoid large memory allocations,
- return scalar and reduced-vector features only,
- skip safely when resources are insufficient.

Add tests for:

- disabled by default,
- compact output,
- no large arrays,
- failure handled as `partial_success`,
- JSON serialization.

FASE 6 — Representación ML-ready

Objetivo

Construir un vector numérico plano, versionado y estable para entrenamiento futuro de modelos.

Archivo

```
analysis_engine/stage6_ml_ready.py
```

Requisitos

El vector debe combinar, según disponibilidad:

- métricas de calidad,
- métricas básicas,
- métricas temporales,
- métricas espectrales,
- métricas cepstrales,
- métricas tiempo-frecuencia compactas.

Salida esperada

```
"ml_ready": {  
  "enabled": true,  
  "schema_version": "ml_vector_v1",  
  "feature_vector": [0.034, 0.01, 0.08, 0.2, 0.5, 0.3],  
  "feature_names": [  
    "quality.snr_estimate",  
    "basic.rms_mean",  
    "basic.rms_std",  
    "temporal.power_slope",  
    "spectral.energy_low",  
    "spectral.energy_mid"  
  ],  
  "feature_vector_length": 6,  
  "missing_features": []  
}
```

Reglas

- vector plano,
- solo valores numéricos,
- nombres estables,
- schema versionado,
- sin deep learning,
- sin entrenamiento,
- sin PCA por defecto,
- manejar features faltantes.

Instrucción específica para Codex — Fase 6

Implement Phase 6: ML-ready representation.

Build a stable flat numerical vector from previous stages.

Do not train models.

Do not add deep learning dependencies.

Do not use PCA by default.

Implement:

- stage6_ml_ready.py
- stable ordered feature schema
- missing feature handling
- schema versioning
- orchestrator integration

Requirements:

- feature_vector and feature_names must align,
- vector must contain numeric values only,
- output must be JSON serializable,
- optional stages must not break the vector builder,
- include missing_features.

Add tests for:

- feature vector length consistency,
- feature_names alignment,
- missing optional stage,
- stable ordering,
- JSON serialization.

FASE 7 — Exportación científica bajo demanda

Objetivo

Permitir exportar datos científicos para investigación sin hacer pesado el endpoint normal.

Archivo

```
analysis_engine/stage7_research_export.py
```

Formatos

Soportar:

```
.mat  
.csv  
.json
```

Principio clave

El endpoint normal no debe devolver datos brutos ni matrices grandes.

El modo research debe generar archivos externos y devolver rutas o referencias.

Contenido recomendado del `.mat`

Incluir, según configuración:

- audio normalizado,
- sample_rate,
- metadatos,
- configuración de tramas,
- features globales,
- features por trama,
- resumen temporal,
- resumen espectral,
- resumen cepstral,
- resumen tiempo-frecuencia compacto,
- vector ML-ready.

No incluir por defecto:

- audio de cada trama,
- matrices CWT completas,
- espectrogramas completos,
- imágenes.

Configuración

```
RESEARCH_EXPORT_CONFIG = {  
    "enabled": False,  
    "formats": ["mat", "json", "csv"],  
    "max_export_size_mb": 50,  
    "include_raw_audio": True,  
    "include_frame_audio": False,  
    "include_full_cwt": False,  
    "max_frames_export": 200,  
}
```

JSON esperado

```
"exports": {  
    "mat": {  
        "enabled": true,  
        "path": "/storage/exports/session_id.mat",  
        "size_mb": 12.4,  
        "included": ["audio", "features", "frame_features", "ml_ready"]  
    }  
}
```

Instrucción específica para Codex — Fase 7

Implement Phase 7: research export.

This must not run by default in the normal analysis endpoint.

It must be explicitly enabled or called through an export flow if the current project already has one.

Do not break existing API behavior.

Implement:

- stage7_research_export.py
- export to .mat using `scipy.io.savemat`
- compact JSON export
- CSV export for frame-level features
- safe export path handling
- size limits
- metadata for reproducibility

Requirements:

- do not include raw frame audio by default,
- do not include full CWT by default,
- return file references, not file contents,
- enforce max export size,
- include schema version and config metadata,
- document research mode behavior.

Add tests for:

- .mat file creation,
- CSV creation,
- JSON creation,
- export size limits,
- excluded heavy arrays by default,
- metadata presence.

12. `dashboard_ready`

Aunque no se construya frontend ahora, la API debe devolver datos listos para cualquier dashboard.

Estructura recomendada:

```
"dashboard_ready": {
  "rms_trend": [
    {"time": 0.0, "value": 0.031},
    {"time": 1.0, "value": 0.035}
  ],
  "energy_trend": [
    {"time": 0.0, "value": 0.002},
```

```

    {"time": 1.0, "value": 0.003}
  ],
  "energy_band_summary": {
    "low": 0.2,
    "mid": 0.5,
    "high": 0.3
  },
  "key_indicators": {
    "duration_seconds": 34.7,
    "snr_estimate": 18.2,
    "stability_index": 0.83,
    "variability_index": 0.21,
    "dominant_frequency": 740.0
  }
}

```

Reglas:

- tendencias reducidas,
- máximo 300 puntos,
- valores escalares,
- no arrays grandes,
- claves estables.

13. Configuración global sugerida

```

ANALYSIS_ENGINE_CONFIG = {
  "version": "0.1.0",
  "mode": "normal",

  "audio": {
    "target_sample_rate": 16000,
    "mono": True,
    "normalize_amplitude": True,
    "max_audio_duration_seconds": 180,
    "accepted_extensions": [".wav", ".webm", ".mp3", ".m4a", ".ogg",
    ".flac", ".aac"],
  },

  "framing": {
    "enabled": True,
    "frame_duration_seconds": 2.0,
    "hop_duration_seconds": 1.0,
    "allow_overlap": True,
    "auto_frame_duration": True,
    "max_frames": 300,
  },
}

```



```

"stages": {
  "enable_stage0_quality": True,
  "enable_stage1_basic": True,
  "enable_stage2_temporal": True,
  "enable_stage3_spectral": True,
  "enable_stage4_cepstral": True,
  "enable_stage5_timefreq": False,
  "enable_stage6_ml_ready": True,
  "enable_stage7_research_export": False,
},

"performance": {
  "max_returned_trend_points": 300,
  "max_wavelet_scales": 64,
  "use_float32": True,
  "avoid_large_arrays_in_json": True,
},

"research_export": {
  "enabled": False,
  "formats": ["mat", "json", "csv"],
  "max_export_size_mb": 50,
  "include_raw_audio": True,
  "include_frame_audio": False,
  "include_full_cwt": False,
  "max_frames_export": 200,
},
}

```

14. Consideraciones regulatorias y de posicionamiento

La API debe usar lenguaje de investigación y bienestar.

Claims permitidos

- Acoustic signal analysis.
- Research-use acoustic feature extraction.
- Exploratory physiological audio analysis.
- Dashboard-ready acoustic trends.
- ML-ready acoustic feature generation.
- Longitudinal acoustic monitoring for research.

Claims prohibidos

- Diagnosis.
- Disease detection.
- Treatment recommendation.
- Clinical decision support.
- Medical triage.

- Predicts illness.
- Detects obstruction.
- Detects disease.

Disclaimer recomendado

This platform is intended for research, wellness, and exploratory signal analysis purposes only. It does not provide medical diagnosis, treatment recommendations, or clinical decision-making support.

15. Roadmap de implementación

Fase A — Base robusta

- Fase 0.
- Validación.
- Normalización.
- Framing.
- Calidad.

Fase B — Métricas ligeras útiles

- Fase 1.
- Fase 2.
- Dashboard-ready básico.

Fase C — Métricas espectrales y cepstrales

- Fase 3.
- Fase 4.
- Vectorización parcial.

Fase D — Tiempo-frecuencia compacto

- Fase 5 opcional.
- Control de memoria.

Fase E — ML-ready

- Fase 6.
- Vector estable.
- Schema versionado.

Fase F — Research export

- Fase 7.
- `.mat`, `.csv`, `.json` bajo demanda.

16. Criterios de éxito

El desarrollo será satisfactorio si:

1. La API actual sigue funcionando.
2. El JSON actual no se rompe.
3. Se añade `analysis_engine` como extensión.
4. El audio se valida y normaliza correctamente.
5. El audio se procesa por tramas.
6. Se aceptan los formatos definidos.
7. Las métricas devueltas son compactas y útiles.
8. Stage 0-4 funcionan en modo normal.
9. Stage 5 es opcional y seguro.
10. Stage 6 produce un vector ML-ready estable.
11. Stage 7 exporta `.mat` solo bajo demanda.
12. No se devuelven arrays grandes en JSON.
13. El sistema funciona en Docker con 8GB RAM.
14. Los resultados son útiles para dashboard e investigación.
15. El sistema no contiene claims diagnósticos.